

Sommario

1.0	Note generali	1
2.0	Processori di tipo vettoriale.....	3
3.0	Tre tipi di parallelismo:	5
4.0	Coerenza dei sistemi a memoria condivisa	6
4.1	UMA- Uniform Memory Access	6
4.2	NUMA- Non uniform Memory Access	6
4.3	Message Passing	6
4.4	Coerenza e consistenza.....	6
4.5	Protocollo di coerenza della cache: Snooping	8
	Snooping con tecnica del write-through.....	8
	3.5.1 Snooping con tecnica del write-back.....	9
5.0	Esempio matrice	10
6.0	Architettura e modello di programmazione del MIPS.....	12
2.1	Modello di programmazione.....	14
2.2	Pipeline del MIPS.....	21

1.0 Note generali

Esistono meccanismi differenti per il parallelismo:

1. **Ottimizzazione delle prestazioni:** Il programmatore lavora per migliorare le prestazioni> concetto di *Speedup*.
 - L'architettura **SIMD** (Single Instruction Multiple Data) risolve il parallelismo sui dati.
2. **Scrittura parallela dei programmi:** Il programmatore può scrivere in modo parallelo dei programmi.
 - Architetture **SPMD** (Single Program Multiple Data).
3. **Gestione delle richieste:** Ci sono applicazioni che non devono velocizzare un singolo programma, ma la gestione di molteplici richieste simultanee.

Modelli Ibridi:

Alcuni modelli finiscono per fondersi. Ad esempio, raggruppando diversi flussi **SIMD**, si può ottenere un'architettura **MIMD** (Multiple Instruction Multiple Data), in cui i vari componenti potrebbero eseguire operazioni o "fare cose" diverse contemporaneamente.

Nelle macchine vettoriali la produttività è alta. Se si dispone di diverse *pipe* (pipeline), si può pensare di gestire **Thread differenti**. In questo modo non si fa solo SIMD, ma si eseguono anche Thread diversi.

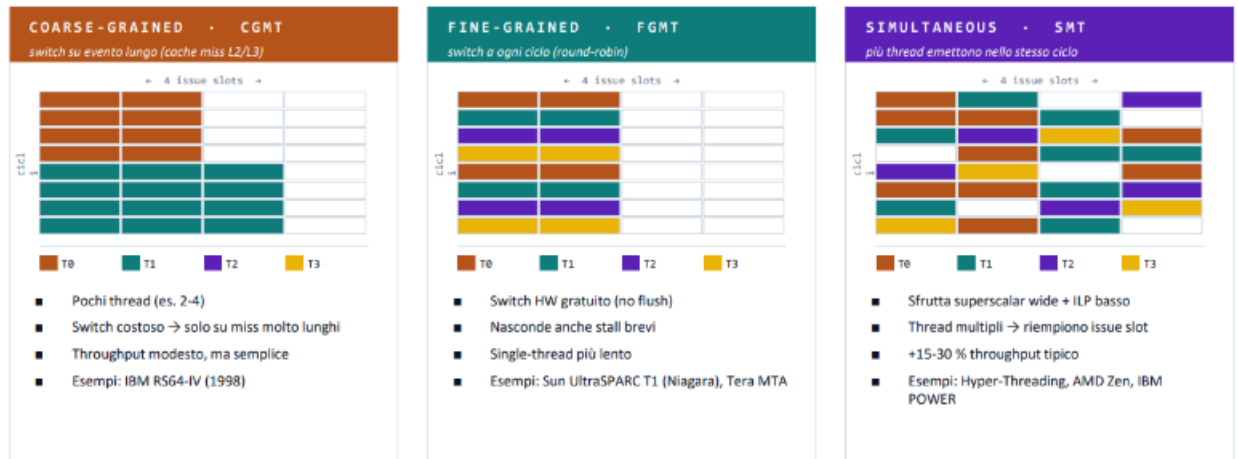
I **Thread** sono "filamenti" di programmi attivi su un determinato nodo. Il processore li può gestire e utilizzare in vari modi:

1. **Switch su Cache Miss:** Mandando avanti un Thread fino a che non si verifica un *cache miss*. A quel punto si fa un cambio di contesto (*context switch*) e si passa a un secondo Thread. Questo meccanismo è molto simile al sistema di gestione delle interruzioni.
2. **Divisione delle risorse:** Dividendo le pipe tra *Task* differenti.

3. **Esecuzione interallacciata:** Utilizzando le risorse in modo interallacciato. Se un Task non può utilizzare tutte le pipe disponibili, queste vengono affidate a un altro Thread.

Tre granularità per condividere una pipeline

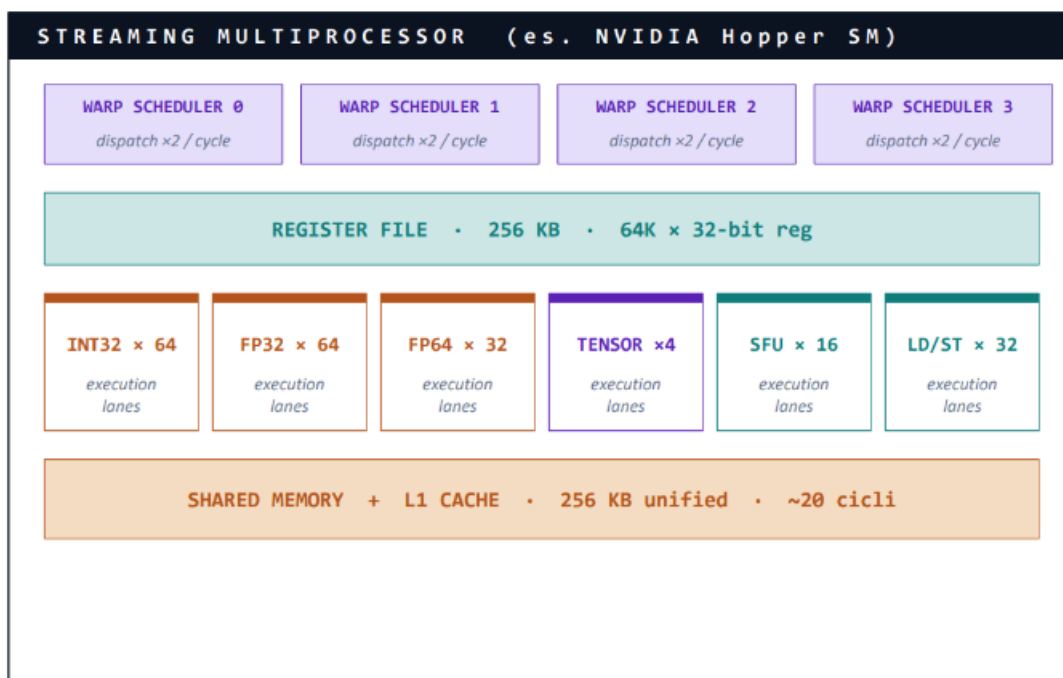
Differiscono per quando avviene lo switch tra thread e quanti thread possono emettere per ciclo



Le moderne GPU funzionano esattamente con questo principio: sono costituite da tanti nodi, ognuno operante in logica SIMD, e hanno più unità elaboratrici moltiplicate in modo da aumentare drasticamente la banda. Avendo così tanti Thread in esecuzione contemporanea, le latenze vengono coperte e il sistema non si ferma praticamente mai.

Architettura di uno streaming Multiprocessor

È il mattone fondamentale delle moderne Gpu NVIDIA. Viene utilizzato il paradigma SIMT (single instruction, multiple Threads). I thread non vengono gestiti singolarmente, ma a gruppi di 32 chiamati WARP.



Ci sono 4 Warp Scheduler e quindi possiamo gestire attivamente le istruzioni di 4 warp diversi contemporaneamente. Ogni warp può lanciare fino a 2 istruzioni indipendenti per ciclo di clock per un particolare warp (per questo c'è $\text{dispatch } \times 2 / \text{cycle}$).

C'è un register file che è un blocco da 256 k (64 mila registri a 32 bit).

Poi abbiamo le lane di esecuzione. Cioè differenti unità funzionali. Ognuna specializzata in operazioni differenti.

1. Cosa significa "INT32 x 64"?

Quel "x64" nel diagramma significa letteralmente che all'interno di quel riquadro ci sono **64 calcolatrici indipendenti** (64 ALU fisiche). (è un blocco SIMD)

Nel gergo GPU, **1 ALU = 1 Execution Lane**. Quindi ci sono 64 corsie di esecuzione parallele.

2. Il momento dell'esecuzione

l'hardware NVIDIA non manda mai in esecuzione un thread da solo, ma muove sempre l'intero **Warp (32 thread)**.

In un singolo ciclo di clock, ecco cosa succede esattamente:

- Il Warp Scheduler decide che il "Warp A" deve fare un'addizione tra numeri interi.
- Lo scheduler prende i **32 thread** del Warp A e li posiziona su **32 delle 64 lane** del blocco INT32.
- La mappatura è strettamente 1 a 1: **Thread 0 -> Lane 0, Thread 1 -> Lane 1**, e così via fino alla Lane 31.
- In quel ciclo di clock, 32 ALU lavorano simultaneamente facendo 32 somme diverse.

3. Cosa succede alle altre 32 lane vuote?

Dato che il blocco ha 64 lane in totale e un Warp ne ha occupate solo 32, l'hardware è progettato per non sprecare nulla.

ogni Warp Scheduler ha la dicitura *dispatch x2 / cycle*. Questo significa che nello stesso identico istante, lo scheduler può prendere un **Warp B** (altri 32 thread) e inviarlo sulle **32 lane rimanenti**.

Risultato? In un solo "tic" dell'orologio del processore, l'unità INT32 lavora a pieno regime eseguendo **64 thread contemporaneamente** su 64 lane separate.

Grazie a un gigantesco **Register File**, l'SM mantiene residenti decine di Warp. Poiché le unità di accesso alla memoria (LD/ST) sono hardware fisicamente separate dalle ALU, quando un Warp effettua una *Load* e va in stallo per la dipendenza dei dati, lo Scheduler effettua un *context switch* a costo zero, riempiendo istantaneamente il vuoto sulle ALU con un altro Warp pronto dalla "panchina". Questo continuo interallacciamento assicura che le risorse matematiche lavorino al 100% senza mai fermarsi in attesa della RAM.

2.0 Processori di tipo vettoriale

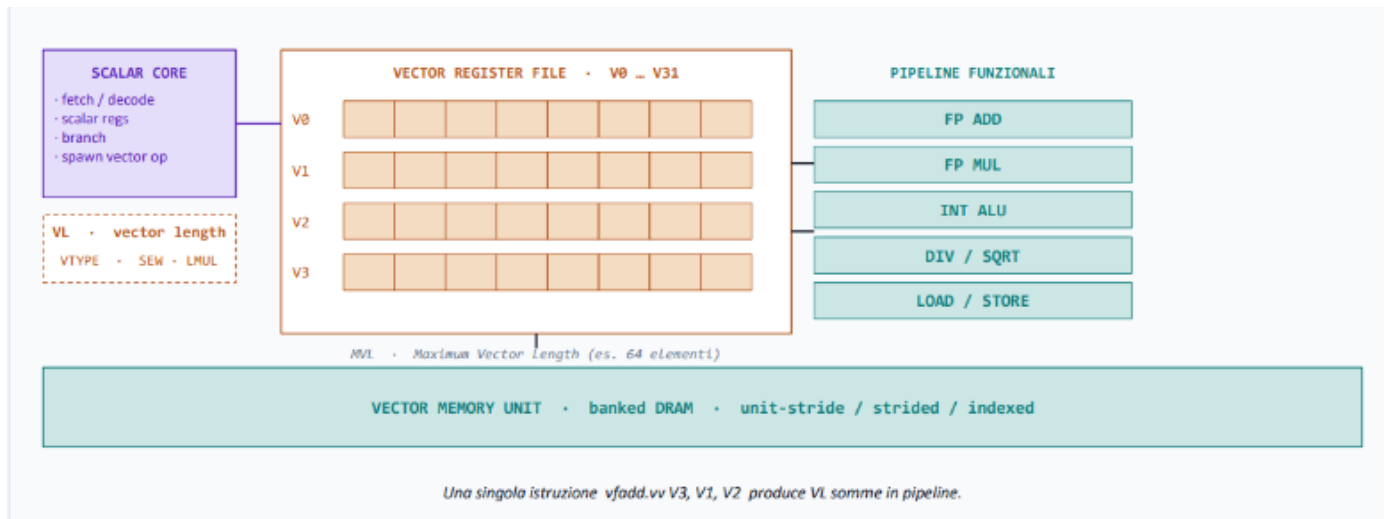
Nelle architetture Packed SIMD abbiamo dei limiti:

- allineamento in memoria: i vettori devono essere allineati in modo che serva una singola lettura e non più letture se non sono allinati
- molto spesso si usano loop per elaborare array di dati. la vettorizzazione è il tentativo del compilatore di prendere le iterazioni del loop e farle eseguire tutte insieme al modulo SIMD.

Sono i primi processori che hanno implementato delle azioni su **vettori di lunghezza arbitraria**. Nelle ottimizzazioni di un vettore la **fase IF e ID vengono fatte un numero minore di volte**. L'idea di base quindi è quella delle architetture SIMD.

Ci sono alcuni componenti che articolano questi processori:

- **Scalar Core** che esegue fetch, decode, gestione dei branch e dei registri scalari.
- **Vector register file**: un insieme di registri vettoriali tipicamente 32, ciascuno profondo MVL elementi (maximum vector length). MVL può valere ad esempio 64. In questo modo ogni registro è effettivamente un vettore lungo 64 elementi.
- **Pipeline funzionali**: unità specializzate in addizione in floating point, moltiplicazione FP, integer ALU, load/store. Ciascuna è pipelined.
- **Vector memory unit**: interfaccia verso la DRAM
- Utilizzano inoltre alcuni registri come **VLR** in cui viene inserita la lunghezza dei vettori corrente.



Consideriamo l'istruzione `vfadd.vv v3,v1,v2`

- V sta per vector
- fadd sta per floating-point add
- .vv significa vector-vector ovvero specifica che entrambi gli operandi sorgenti sono vettori

Questa istruzione prende gli elementi del vettore V1 e V2, li somma e li mette in V3.

Il parametro **vector Length** serve per fare in modo che bisogna prendere solo una parte del registro vettoriale e quindi se i vettori sono lunghi 50 farò 50 somme, è inutile fare 100 somme (se i registri sono lunghi 100).

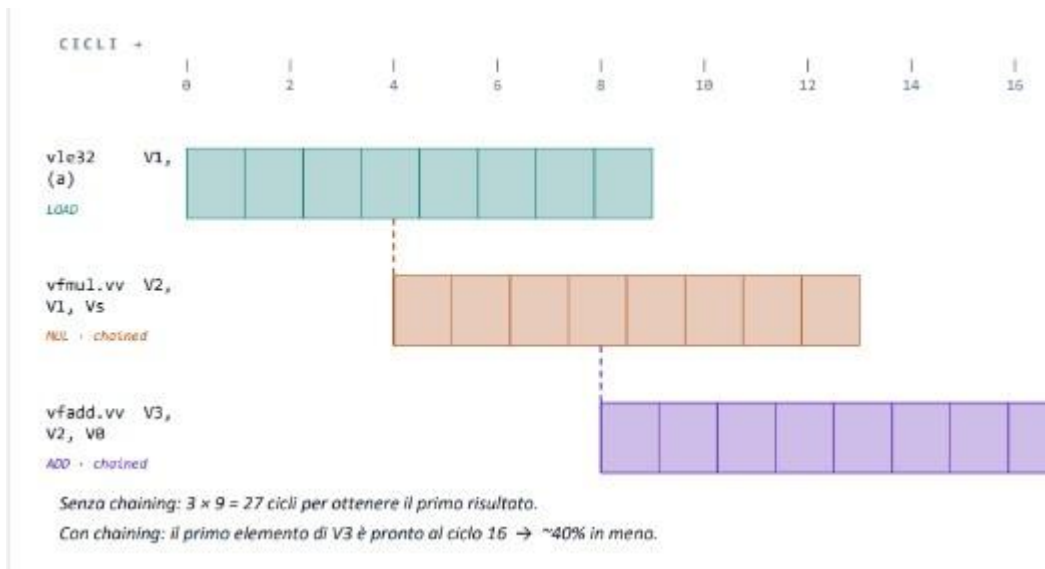
Inoltre se consideriamo che dobbiamo leggere, sommare e scrivere ogni coppia di elementi. Quindi 3 cicli di **clock x VL**. In realtà si sfrutta un **meccanismo di pipeline** e cioè mentre si legge una coppia, se ne somma un'altra e se ne scrive un'altra.

Poi abbiamo il meccanismo di **chaining**: senza chaining ogni istruzione vettoriale aspetterebbe che la precedente termini tutti i suoi VL cicli. Con il chaining la seconda istruzione partirebbe qualche ciclo dopo la prima e non dopo VL cicli.

Consideriamo la sequenza di istruzioni

- LOAD V1
- MUL: moltiplica V1 con V5 e salva in V2
- ADD: somma V2 con un altro e salva in V3

* Nelle SIMD le 8 ALU lavorano esattamente nello stesso istante (ciclo di clock o stadio di pipeline). Nei veri processori vettoriali, invece, un'istruzione può specificare una lunghezza variabile del vettore e i dati fluiscono attraverso unità funzionali in pipeline su più cicli.



Il chaining consente di sovrapporre le pipe. La load inizia a caricare. Al ciclo 4 il primo elemento di V1 è pronto. Invece di parcheggiarlo in attesa di caricare tutto, l'hardware forwarda questo singolo elemento alla pipeline della moltiplicazione. Quindi abbiamo che la LOAD sta caricando il secondo elemento e nel mentre la MUL già sta moltiplicando il primo elemento

Nelle architetture SIMD si utilizzano delle maschere di predicato che disabilitano delle lane individuali quando dobbiamo gestire delle condizioni if/else. Questo però porta a sprecare dei cicli.

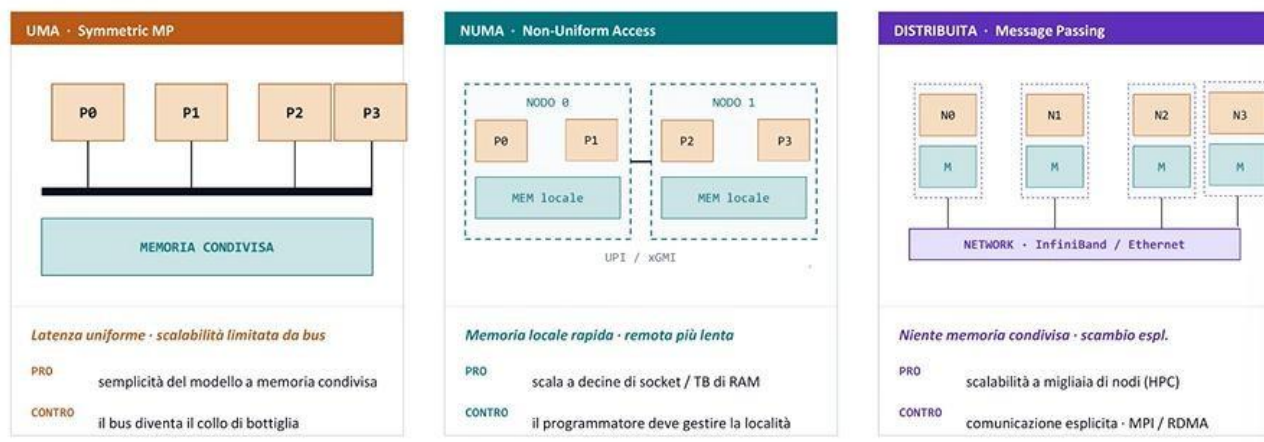
Una pipe non migliora il tempo di esecuzione di un'istruzione, ma migliora la produttività.

Nei processori la produttività aumenta ancora di più.

3.0 Tre tipi di parallelismo:

- **ILP a livello di istruzione**: quante istruzioni possono essere eseguite simultaneamente. Il processore identifica le istruzioni indipendenti e può eseguirle in parallelo o scambiandole tramite meccanismi di associatività oppure internal forwarding.
- **DLP a livello di dato**: Eseguire la stessa istruzione su più dati contemporaneamente. Principio SIMD
- **TLP a livello di thread**: sfrutta l'esecuzione parallela di diversi thread spesso su processori multi-core.
- **RLP a livello di richieste**: nasce dalla gestione simultanea di un gran numero di richieste. È il regno dei data center.

4.0 Coerenza dei sistemi a memoria condivisa



Abbiamo 3 modelli di memoria che consentono di attuare tecniche differenti per la programmazione concorrente.

4.1 UMA- Uniform Memory Access

Nel modello UMA tutti gli oggetti accedono alla memoria allo stesso modo. Quindi la memoria è condivisa tra i vari processori e siamo nel modello multiprocessore. Il programmatore può ragionare in termini di variabili condivise e mutue esclusioni. Il problema è la scalabilità.

4.2 NUMA- Non uniform Memory Access

Il modello NUMA prevede che ogni processore abbia la propria memoria ed è molto più scalabile. La memoria diventa geografica. La particolarità di questo modello è che si può accedere anche alla memoria di altri nodi, quindi c'è bisogno di un'attenzione esplicita. Parliamo di non uniforme perché i tempi di accesso alla memoria ora sono differenti e dipende da dove si trova il dato rispetto a chi lo sta chiedendo.

4.3 Message Passing

Un terzo tipo hanno la memoria non condivisa, ma di tipo privato. L'unico modo per poter interagire è tramite una rete. Siamo nel campo del multicomputer. Ogni nodo possiede il proprio sistema operativo, la propria RAM, la propria gerarchia di cache. Il coordinamento avviene per scambio esplicito di messaggi attraverso la rete di interconnessione.

Se abbiamo due istanze di kubernetes la comunicazione non avviene tramite shared memory, ma tramite scambio di messaggi, poiché è più vero che è più lento, però non da problemi di conflitti sulla memoria.

4.4 Coerenza e consistenza

- Coerenza di cache:** riguarda una singola locazione di memoria alla volta. Se più cache contengono copie dello stesso indirizzo, come garantiamo che la scrittura effettuata da un core diventi visibile agli altri? La soluzione richiede due proprietà:
 - Write propagation:** la modifica raggiunge tutte le copie
 - Write serialization:** tutti i core osservano le scritture allo stesso indirizzo.
- Modello di consistenza:** riguarda l'ordine globale in cui molti accessi a indirizzi diversi diventano visibili ai diversi thread. Devo garantire che uno solo possa scrivere in una locazione di

memoria tramite il concetto di semaforo. Deve essere fatto in hardware.

un sistema può essere coerente (tutti vedono lo stesso valore per ogni indirizzo) e tuttavia adottare un modello di consistenza rilassato (le scritture a indirizzi diversi diventano visibili in ordini diversi a thread diversi)

Tutti quanti vedono la variabile x nella memoria condivisa. I due processori per andare in memoria farebbero sempre dei conflitti. Sarebbe peggio del DMA.

I sistemi sono realizzati mediante due cache. Ci sono tanti core ognuno ha la sua cache e poi c'è una memoria condivisa. Ognuno avrà delle variabili locali. Quando però entrambi leggono il dato x che è condiviso e finché lo leggono non c'è problema. Se però P_i e P_j leggono il dato e poi P_i lo modifica significa che P_j poi avrà il dato sbagliato.

Se utilizziamo una tecnica di tipo write-through quando x diventa x' nella cache di P_i , anche nella memoria condivisa x diventa x' . Il problema è che P_j ha il dato x . Se ora pensa di leggere x , legge x di una copia precedente. Se la tecnica è write back il problema è anche peggio perché la memoria continua ad avere il dato vecchio.

4.5 Protocollo di coerenza della cache: Snooping

Nei sistemi multicomputer e multiprocessore l'obiettivo è quello di diminuire i costi di interazione senza agire sul software, ma solo sull'hardware. Il costo di interazione è il carico introdotto sull'elaborazione, dalla comunicazione, per lo scambio di informazioni. Per un sistema multiprocessore tale costo è determinato dal costo a cui è soggetto il bus comune dai processi che interagiscono per mezzo della memoria condivisa.

Al fine di diminuire l'interazione tra i processi è necessario introdurre all'interno dell'architettura una gerarchia di memoria che si traduce nell'assegnare ad ogni processore una propria memoria (cache di solito) - Sistemi Multicore.

Nasce quindi un problema di coerenza della memoria condivisa, non per la memoria istruzione visto che sono soltanto lette, ma per i dati che sono condivisi e possono essere modificati.

Bisogna trovare delle soluzioni che valgano sia in caso di uso della tecnica write-through che write-back.

Si necessita di una politica di invalidazione dei dati modificati da un processo ed eventualmente presenti all'interno delle cache di altri processori, o della memoria comune.

Se usassimo la tecnica del write through, la memoria condivisa viene aggiornata immediatamente dalla cache e quindi non possiede mai dati da invalidare.

Ogni processore spia gli altri per verificare se un dato presente nella propria cache viene modificato. Un dato viene identificato dall'indirizzo in memoria condivisa. Poiché la realizzazione dell'hardware necessario per il meccanismo di invalidazione è molto complesso, il numero di processori è molto limitato (2 max 4).

Snooping con tecnica del write-through

In questo caso visto che le modifiche sono istantanee (la cache ricopia subito in memoria condivisa le modifiche apportate), la spia viene effettuata sulle linee indirizzo del bus comune.

Consideriamo due processori P_i e P_j che operano sugli stessi dati. L'automa descritto rappresenta il comportamento del processore P_i (processore SPIA) a fronte delle azioni compiute sui dati, sia da se stesso, che dal processore P_j (processore spiato).

Ogni dato presente in cache assume due valori: Valido e Non valido. R -> lettura, W-> scrittura, Z -> il processore elimina il dato dalla propria memoria locale.

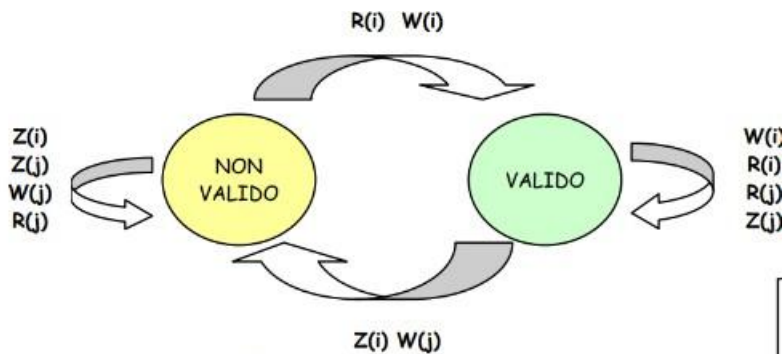


Figura 64: Automa nel caso Write-Through.

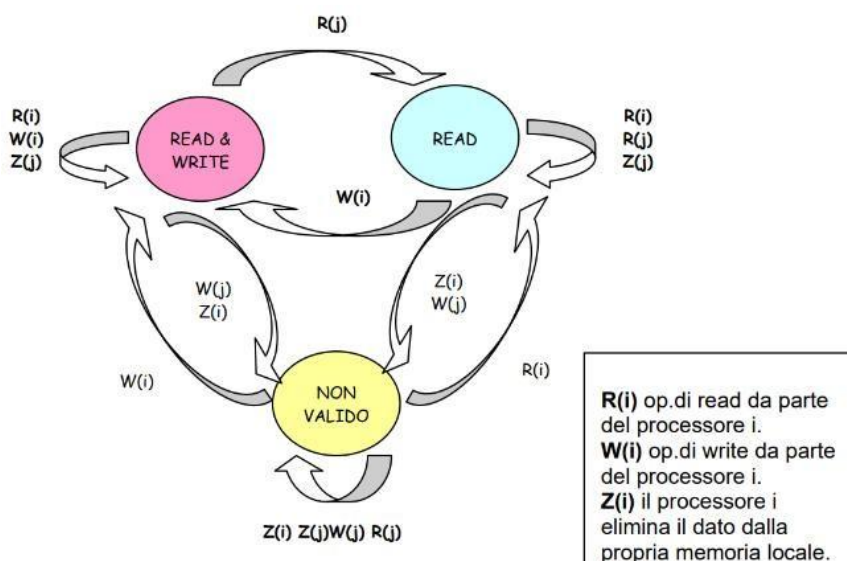
- Tutte le operazioni di lettura e scrittura effettuate da P_i fanno rimanere (oppure portano) i dati validi. Inoltre in tale stato la lettura o la rimozione di altri fanno rimanere validi i dati (se P_j non li modifica oppure li elimina dalla sua cache non vedo perché dovrebbero invalidarsi).
- Si passa nello stato di non validità appena P_i elimina il dato (operazione $Z(i)$) perché sta a indicare che quel dato non lo ha più e se lo riuole deve rileggerlo dalla memoria condivisa, oppure se P_j lo modifica. Una volta che siamo in questo stato ulteriori operazioni di lettura e scrittura di altri processori fanno rimanere in questo stato così come operazioni di rimozione da parte di entrambi.

3.5.1 Snooping con tecnica del write-back

In questo caso bisogna considerare il fatto che l'aggiornamento del dato in memoria comune non avviene subito, ma in un secondo momento.

Lo stato RW è quello in cui P_i può effettuare qualunque tipo di operazione sul dato perché sta lavorando con il suo dato e non lo ha portato ancora in memoria.

Se P_j legge si va nello stato READ perché si deve tener conto che altri hanno letto il dato visto che non lo avevo portato in memoria e quindi si deve effettuare un flush in memoria comune. Da questo stato non si esce finché i vari processori continuano a leggere il dato.



Se P_i scrivesse di nuovo torniamo nello stato RW sempre per il fatto che rimane locale alla cache. Dal punto di vista di P_j invece corrisponde a invalidare il dato.

Sia che ci troviamo in RW che in READ, se un altro processore scrivesse la locazione, corrisponde a invalidare il dato dal punto di vista della SPIA P_i. Questo significa che con la tecnica write-back la scrittura di un processore renderà invalido il dato per tutti gli altri processori.

Quindi anche se ho modificato dei dati in cache, e qualcun altro li modifica, devo necessariamente dire che non è valido perchè le mie modifiche non le ho riportate nella memoria condivisa.

Se a questo punto leggessi o scrivessi posso validare di nuovo il dato. La lettura mi riporta in READ, mentre la scrittura in RW perché scrivendo ho di nuovo quelle modifiche solo per me, mentre leggendo costringo l'altro processore a ricopiare in memoria condivisa le sue modifiche (come quando io P_i sto in RW e l'altro legge).

Lo stato più critico è quello di RW dove i dati modificati da P_i sono validi solo per stesso e non allineati alla memoria condivisa.

Poiché non sappiamo quando un processore modificherà il dato il tutto è fatto via hardware.

Con il DMA non abbiamo bisogno dello snooping. È attivato via software, è deterministico. Il SO sa esattamente quando il DMA inizia a trasferire e in quale blocco di indirizzi andrà a leggere o scrivere. L'unica cosa da fare quindi è invalidare i dati nella cache.

5.0 Esempio matrice

Abbiamo due matrici A e B 4x4

Il sistema è SPMD, e abbiamo 4 unità SIMD ognuna delle quali ha una propria memoria, un processore e dei dispositivi di I/O. il sistema quindi è a memoria distribuita ovvero comunicano tramite i dispositivi di I/O.

Vogliamo parallelizzare il programma che effettui il prodotto tra le matrici in modo da renderlo più veloce.

$$A = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 1 & 1 & 2 & 0 \end{pmatrix}$$
$$B = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}$$

la matrice C avrà sulla prima riga come primo elemento il prodotto tra prima Riga di A e prima colonna di B, come secondo elemento prima riga di A e seconda colonna di B

capiamo quindi che ogni riga ha bisogno di tutta la matrice B e una sola riga di A

Possiamo dare ad ogni nodo l'intera matrice di B e poi una colonna diversa di A.

Le unità non devono scambiarsi dati quindi $T_{int}=0$. Lo speedup allora è 4. Potremmo pensare di aumentare il numero di unità per aumentare lo speed up non considerando però che l'occupazione di memoria sarebbe elevata. Con 4 nodi devo memorizzare tutta A e poi 4 volte B perché la replico sui 4 nodi.

Ipotizziamo allora di essere nel caso di due unità. Al nodo 1 do riga 1 e riga 2 di A e riga 1 e riga 2 di B, al nodo 2 riga 3/4 di A e riga 3/4 di B.

Il nodo 1 quindi calcolerà le prime due righe della nuova matrice C e il secondo nodo la riga 3 3 4. Però i singoli nodi hanno necessariamente bisogno delle altre due righe di B per completare l'operazione e quindi è previsto uno scambio di messaggi tramite i dispositivi di I/O, di conseguenza avremo un Tempo di interazione che non è più zero.

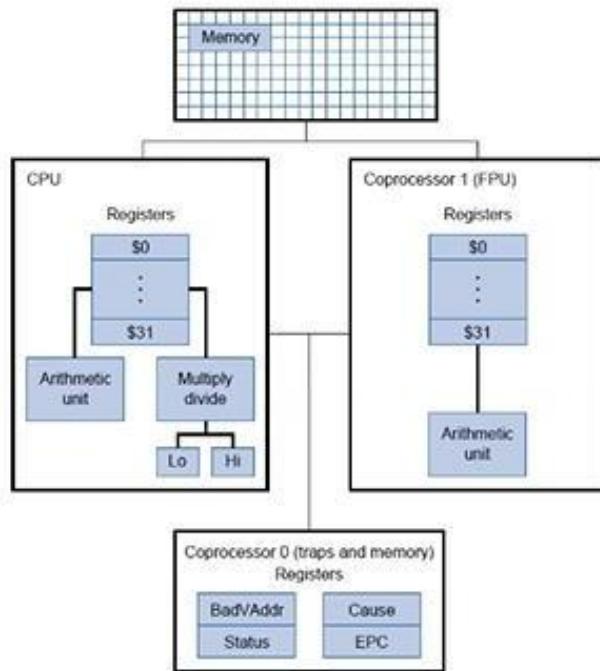
Ma sarebbe stato più efficiente con 2 o 4 processori?

Con 4 processori ognuno avrebbe avuto una sola riga di A e una sola colonna di B, ma in questo modo non potrei sfruttare il tempo morto in cui aspetto di prendere i dati dalla memoria, mentre con 2 unità ognuno ha 2 thread e mentre uno prende una riga dalla memoria, l'altro fa già i calcoli.

6.0 Architettura e modello di programmazione del MIPS

Il MIPS è il processore da cui sono stati ricavati l'arm, il riscv ed è composta da CPU + insieme di coprocessori (fino a 4).

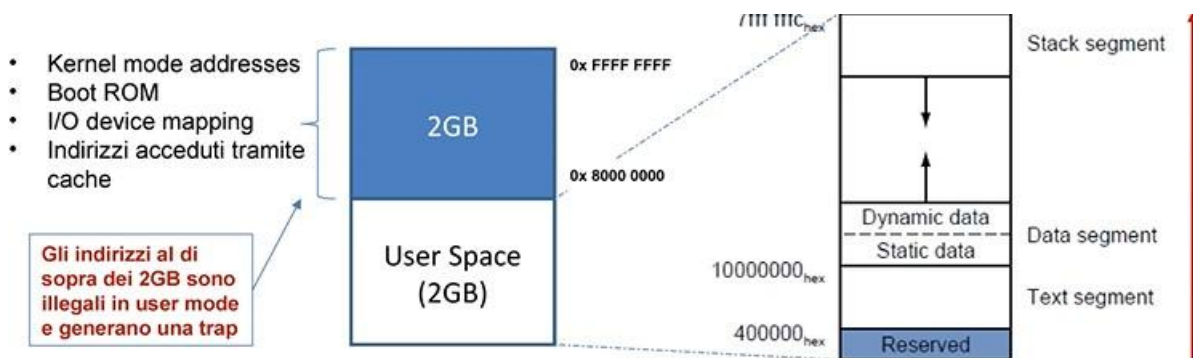
È un'architettura RISC, la cpu dispone di 32 registri general purpose a 32 bit che sono indicati con \$ da 0 a 31.



- 1) Il coprocessore 0 contiene le informazioni necessarie per la gestione di eccezioni ed interruzioni e contiene lo status register.
- 2) Il coprocessore 1 è la floating-point unit e consiste in 32 registri a 32 bit per operazioni in virgola mobile. Sono indicati dal simbolo di dollaro e poi FP0 fino a FP31.

Abbiamo due modalità operative: user e kernel mode. Questa cosa è fondamentale soprattutto nell'accesso alla memoria infatti i task utente non possono accedere alle risorse di memoria allocate per il kernel e non possono modificare lo stato della macchina perché si trova nel coprocessore 0 e non possono accedere.

La memoria è divisa in pezzi:



I 2 GB servono a gestire le risorse in stato supervisore e vanno da 0x80000000 a 0xffffffff.

La memoria utente è divisa in 3 parti:

- **Text segment:** contiene le istruzioni e si trova sopra una porzione di memoria riservata.

- **Data segment:** contiene i dati ed è suddivisa in area dati statici ovvero quegli oggetti la cui dimensione è nota a tempo di compilazione e il tempo di vita è pari al tempo di esecuzione. E poi c'è l'area dati dinamici che viene allocata durante l'esecuzione del programma.
- **Stack segment:** parte dalla cima dell'user space e decresce verso l'area dati quindi verso indirizzi più bassi.

I processori MIPS operano sia in modalità big endian che little endian.

A differenza delle architetture CISC, che spesso salvano automaticamente gran parte del contesto sullo stack, il MIPS (essendo un'architettura RISC pura) delega gran parte di questo lavoro al software, fornendo solo i meccanismi hardware essenziali tramite il **Coprocessore 0 (CP0)**.

La gestione delle interruzioni è affidata al coprocessore 0. Memorizza le informazioni di cui il software ha bisogno per gestire le eccezioni. Ci sono dei registri che gestisce:

- **BadVAddr:** bad virtual address e riguarda una causa dovuta ad un errore di indirizzamento. Il badvaddr memorizza l'esatto indirizzo che ha causato il crash
- **Status:** contiene i bit di abilitazione delle interruzioni.
- **Cause:** contiene il tipo dell'eccezione. Nello specifico il codice ad esempio 0 per un'interrupt hardware e poi i bit pending che indicano quali linee di interrupt hardware sono attive.
- **EPC:** exception program counter che memorizza l'indirizzo dell'istruzione che è stata interrotta. È fondamentale per poter riprendere l'esecuzione del programma utente una volta gestita l'interrupt.

Number	Name	Cause of exception
0	Int	interrupt (hardware)
4	AdEL	address error exception (load or instruction fetch)
5	AdES	address error exception (store)
6	IBE	bus error on instruction fetch
7	DBE	bus error on data load or store
8	Sys	syscall exception
9	Bp	breakpoint exception
10	Ri	reserved instruction exception
11	CpU	coprocessor unimplemented
12	Ov	arithmetic overflow exception
13	Tr	trap
15	FPE	floating point

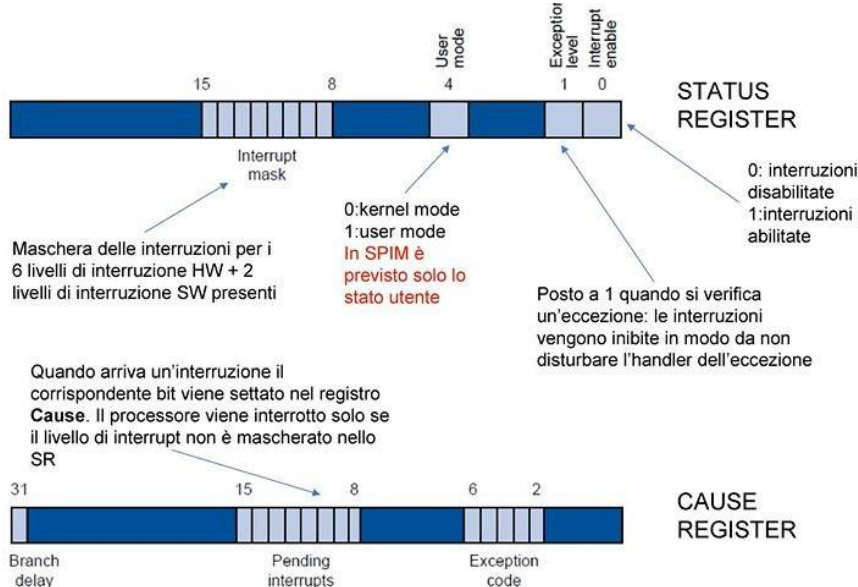
- In seguito ad eccezioni o interruzioni il processore MIPS salta all'indirizzo 80000180hex (nel kernel address space) che contiene l'*exception handler*.
- Il codice a questo indirizzo esamina la causa dell'eccezione e invoca il sistema operativo che effettua le dovute operazioni (arresto di un processo, caricamento di una pagina dal disco alla memoria centrale, lettura o scrittura su I/O).
- Dopo aver gestito l'eccezione/interruzione, l'handler ritorna all'istruzione contenuta in EPC oppure a quella immediatamente seguente

Nel kernel Space c'è un indirizzo in cui si trova l'*exception handler* cioè il vettore delle interruzioni. La differenza con il Motorola è che abbiamo una tabella delle interruzioni e viene calcolato un offset per capire a quale ISR dover saltare. Abbiamo interruzioni vettorizzate e autovettorizzate. Nel MIPS invece abbiamo un singolo punto d'ingresso. Una volta che nel Program counter c'è questo indirizzo, troviamo un gestore generale delle eccezioni. Si legge il registro cause e dai bit 2 a 6 troviamo il codice dell'eccezione. Se il codice è 0 è un'interruzione hardware. Se è hardware deve capire quale periferica l'ha generata e guarda i bit da 8 a 16 ovvero il campo IP cioè le interrupt pending per capire la linea.

Il MIPS utilizza 8 bit per identificare la linea poiché non codifica in binario le linee poiché se usasse 3 bit non saprebbe distinguere tra due richieste in contemporanea. Invece così sa che la linea 0 è quella con 1 in posizione 0. Nel Motorola invece abbiamo un priority encoder e un meccanismo di handshaking sul BUS. Il priority encoder si occupa di gestire le chiamate simultanee facendo arrivare al processore una richiesta per volta.

A questo punto in base al codice della causa si salta alla ISR corrispondente.

Quando si verifica un'interruzione l'hardware forza il bit1 dello SR ad 1 in modo da passare in modalità kernel e disabilitare le interruzioni in modo da ignorare qualsiasi interruzione nuova che arrivi. Questo si fa per evitare che arrivi una nuova interrupt e venga inoltre sovrascritto il registro EPC



Nella cpu abbiamo delle unità hardware che gestiscono soltanto la moltiplicazione e la divisione. Quest'unità si chiama Multiply and divide unit che utilizza due registri a 32 bit HI e LO per memorizzare i risultati.

2.1 Modello di programmazione

Tutti i registri li posso usare in modo libero. Ci sono delle convenzioni su alcuni registri però.

Nome	Numero	Utilizzo	Preservato durante le chiamate
\$zero	0	costante zero	Riservato MIPS
\$at	1	riservato per l'assemblatore	Riservato Compiler
\$v0-\$v1	2-3	valori di ritorno di una procedura	No
\$a0-\$a3	4-7	argomenti di una procedura	No
\$t0-\$t7	8-15	registri temporanei (non salvati)	No
\$s0-\$s7	16-23	registri salvati	Si
\$t8-\$t9	24-25	registri temporanei (non salvati)	No
\$k0-\$k1	26-27	gestione delle eccezioni	Riservato OS
\$gp	28	puntatore alla global area (dati)	Si
\$sp	29	stack pointer	Si
\$s8	30	registro salvato (fp)	Si
\$ra	31	indirizzo di ritorno	No

Oltre a questi registri il modello di programmazione include anche il PC, i registri delle unità floating point e i registri HI e LO.

Un codice in cui dico che registro A=B dovrebbe avere un codice operativo specifico. Ma lo evita facendo un'addizione con un registro. Con quale? Con il registro zero quindi serve per fare quest'assegnazione di registri.

Il registro riservato all'assemblatore serve per le pseudoistruzioni. È utile per scrivere un'istruzione in cui ad esempio carichiamo un indirizzo a 32 bit e non possiamo fare in una sola botta.

In questo processore l'indirizzo di ritorno di una funzione non si salva sullo stack in automatico ma viene salvato nel registro 2-3

- \$at (1), \$k0 (26) e \$k1 (27) sono riservati all'assemblatore e al sistema operativo, non dovrebbero essere utilizzati da programmi utente o compilatori;
- \$a0-\$a3 (4-7) vengono utilizzati per il passaggio dei primi quattro argomenti alle procedure (mentre quelli rimanenti vengono passati mediante lo stack)
- \$v0 e \$v1 (2 e 3) vengono utilizzati per restituire i risultati calcolati dalle funzioni
- \$t0-\$t9 (8-15, 24 e 25) sono salvati dal chiamante e sono utilizzati per contenere valori temporanei che non hanno bisogno di essere salvati dalla procedura;
- \$s0-\$s7 (16-23) sono salvati dal chiamato e contengono i valori che devono essere preservati durante la chiamata e l'esecuzione delle procedure;
- \$gp (28) è il puntatore all'area di memoria del segmento di dati statici;
- \$sp (29) è lo stack pointer che punta all'ultima locazione dello stack;
- \$s8 (o anche \$fp, 30) è il frame pointer ossia il puntatore al frame della procedura;
- \$ra (31) contiene l'indirizzo di ritorno da una procedura (scritto mediante l'istruzione jal)

I registri t sono i registri temporanei mentre s sono i registri saved. La differenza riguarda soltanto una convenzione di chiamata.

Se chiamo un'altra funzione cosa succede ai miei dati nei registri?

Dentro una funzione i registri t possono essere sovrascritti, quindi si è consapevoli che se chiamo una funzione i registri t possono essere sovrascritti e perdo i dati. È il chiamante che si occupa di salvare i dati sullo stack quando chiama una funzione.

I registri s sono i registri sicuri. Una funzione ovvero il chiamato assicura che al termina della funzione, chi ha chiamato la funzione ritroverà gli stessi valori di prima e quindi è il chiamato che si occupa di salvare quei valori.

In generale i registri t li usiamo per mantenere qualcosa di breve durata, mentre i registri s qualcosa di lunga durata.

Le istruzioni del MIPS sono a lunghezza fissa 32 bit e sono word-aligned. Word-aligned significa che gli ultimi due bit del program counter sono sempre 00. Nel MIPS una word sono 4 byte.

Ha 3 tipi di istruzioni:

- R a 3 operandi
- I a 2 operandi
- J a 1 operando

Un codice operativo è sempre codificato sui primi 6 bit quindi abbiamo 2^6 codici operativi. Tuttavia ci sono altre istruzioni che vengono individuate dall'intero indirizzo.

Alcuni codici operativi sono meglio specificati dai primi 6 bit ovvero il campo funct. Le istruzioni possono specificare fino a 3 operandi (destinazione e due sorgenti) codificati su 5 bit.

La convenzione sull'istruzione è <codop dest, sorg1, sorg2>

Ci sono alcune istruzioni fornite dall'assembler MIPS che vengono opportunamente tradotte e vengono chiamate pseudoistruzioni.

I registri del processore essendo 32 vengono codificati su 5 bit.

Istruzioni di tipo R

op	rs	rt	rd	shamt	funct
6 (31:26)	5 (25:21)	5 (20:16)	5 (15:11)	5 (10:6)	6 (5:0)

R type = 3 register operands

• **rs** ed **rt** sono operandi sorgente, mentre **rd** è l'operando destinazione.

- Per alcune operazioni l'operando destinazione o uno degli operandi sorgente potrebbero non essere specificati (es. nella versione a 2 operandi di **MULT** l'operando destinazione non è specificato poiché il risultato dell'operazione viene salvato in appositi registri **HI** e **LO**). In tal caso i bit corrispondenti sono posti a zero.

• Il campo **shamt** (bit 10:6) assume normalmente valore 0 per istruzioni di tipo R, tranne che per alcuni codici di shift, per i quali specifica il numero di operazioni di shift da effettuare (es. **SLL**, **SRL**, **SRA**).

• Il campo **funct** specifica, ove previsto, l'istruzione da eseguire in base al codice operativo (es. per il codice operativo 0, specificando **funct=0x20** si esegue una **ADD**, mentre per **funct=0x21** si esegue una **ADDU**).

Il codice operativo identifica che è un **ADD** però poi con **funct** possiamo specificare il tipo di add come add unsigned.

Istruzioni di tipo R - esempi

- Il tipo R è quello più utilizzato dalle istruzioni aritmetico-logiche:
 - **add rd,rs,rt** : $rd := rs + rt$ con overflow
 - **and rd,rs,rt** : $rd := rs \text{ AND } rt$
 - **div rs,rt** : divide *rs* per *rt* e pone il quoziente in **LO** e il resto in **HI**
 - **mul rs,rt** : moltiplica i registri *rs* e *rt* e pone la parte bassa del prodotto il **LO** e quella alta in **HI**
 -
- Il tipo R viene utilizzato anche per codificare alcune operazioni di movimentazione dati:
 - **move rdest, rsrc** : (pseudo) sposta il contenuto di *rsrc* in *rdest*. Questa pseudo-istruzione viene realizzata come **add rdest,rsrc,0**
 - **movn rd, rs, rt** : sposta il contenuto di *rs* nel registro *rd* se *rt* non è 0
 -

Istruzioni di tipo I

op	rs	rt	imm
6 (31:26)	5 (25:21)	5(20:16)	16(15:0)

I type = 2 register operands e un 16-bit immediate operand

• **rt** è l'operando destinazione, mentre **rs** è l'operando sorgente.

– Per alcuni codici operativi l'operando sorgente può essere nullo (es. LUI)

• **imm** è un immediato espresso su 16 bit. Per operazioni aritmetiche come add, addu, l'immediato viene prima esteso in segno e poi utilizzato.

• L'indirizzamento realizzato è quello **immediato**.

Esempi

• **addi rt,rs,imm**: somma rs con l'immediato imm esteso in segno e pone il risultato in rt

• **andi rt,rs,imm**: effettua la AND tra rs e l'immediato imm esteso con tutti 0 in testa e pone il risultato in rt

• **lui rt, imm**: carica la metà alta di imm nella metà alta di rt, mentre l'altra metà di rt è posta a 0.



Le istruzioni di tipo I consentono anche di effettuare i branch che sfruttano i 16 bit per l'indirizzo

Istruzioni di tipo I e codici branch

op	rs	rt	offset
6 (31:26)	5 (25:21)	5(20:16)	16(15:0)

• Le istruzioni di tipo I vengono utilizzate anche per i **branch**, che sfruttano un **offset** di 16 bit con segno per costruire l'indirizzo dell'istruzione a cui saltare se una data condizione è verificata.

• Con un offset di 16 bit è possibile saltare fino a $2^{15}-1$ istruzioni in avanti o 2^{15} indietro.

– Il campo su cui è possibile spaziare è quindi di $2^{15} * 4 * 2 = 2^{18}$ byte = 256KB

• L'indirizzo dell'istruzione a cui saltare è dato da:

$$A = PC + 4 + 4 * \text{offset}$$

– Si noti che offset rappresenta un indice, e $4 * \text{offset}$ dà l'effettivo spiazzamento in byte.

• L'indirizzamento così realizzato è quello **relativo a PC**.



Istruzioni di tipo I e load/store

op	rs	rt	offset
6 (31:26)	5 (25:21)	5 (20:16)	16 (15:0)

- Le istruzioni **load** e **store** sono di tipo I: il campo offset di 16 bit rappresenta lo spiazzamento da aggiungere al registro rs indicato nell'istruzione per determinare l'indirizzo effettivo dell'operando.
 - **la rdest, address***: (pseudo) carica l'indirizzo address in rdest
 - **lb rt, address***: carica in rt il byte posto all'indirizzo address* esteso in segno
 - **sb rt, address***: memorizza il byte più basso del registro rt alla locazione address*
- L'operando address* viene specificato con *offset(registro)*, e viene calcolato sommando l'offset al valore contenuto in un registro (implicito in assembly)
- L'indirizzamento così realizzato è il **base addressing** (registro base + spiazzamento).

Esempio:

```
.data
V: .byte 5,0,1,3,2
dim: .byte 5
stringa_ris: .asciiz "la somma degli elementi del vettore e': "
RTN: .asciiz "\n"
```

Questa è l'area dati, come notiamo non definiamo un indirizzo come facciamo con ORG nel 68k

```
.text
.globl main

# Programma principale

main:
    la $t0, V           # carica l'indirizzo del vettore V in $t0
    lb $t1, dim          # carica dim in $t1

    li $s0, 0           #metto 0 in $s0 che sarà il contatore degli elementi
    li $s1, 0           #metto 0 in $s1 che sarà la somma
```

la prima istruzione in realtà non è un'istruzione perché l'indirizzo è a 32 bit quindi in realtà sono 2 istruzioni.

I registri sono individuati dal dollaro.

```

ciclo:
    #lb $t2, V($s0)           #carica l'elemento i-esimo di V in $t2
    #add $s1, $s1, $t2        #aggiunge l'elemento i-esimo $t2 alla somma parziale $s1
    #addi $s0, $s0, 1          #aggiunge 1 al contatore
    #bne $s0, $t1, ciclo      #salta a ciclo se $s0<$t1

    # versione alternativa:
    lb $t2, ($t0)             #carica in $t2 l'elemento i-esimo, puntato da $t0. Questo modo di indirizzamento è nativo della
    add $s1, $s1, $t2         #aggiunge l'elemento i-esimo $t2 alla somma parziale $s1
    addi $t0, $t0, 1          #fa avanzare il puntatore di un byte per puntare al byte successivo
    addi $s0, $s0, 1          #aggiunge 1 al contatore
    bne $s0, $t1, ciclo      #salta a ciclo se $s0<$t1

    #scrittura del risultato in $v0
    move $v0, $s1             #pseudo istruzione che sposta il risultato da $s1 a $v0

    #chiamate aggiuntive che sfruttano le system call
    #per la visualizzazione del risultato: in $v0 viene posto sempre il codice
    #della system call da invocare, in $a0-$a3 gli argomenti

    #stampa messaggio per il risultato (servizio #4)
    la $a0, stringa_ris
    li $v0, 4                 #4 è il codice per print_string
    syscall

```

La prima versione usa un modo di indirizzamento che è base + spiazzamento quindi una pseudoistruzione

Salviamo la somma in un registro v0 che è un registro atto ad ospitare i valori di ritorno delle funzioni.

```

# stampa somma (servizio #1)
move $a0, $s1
li $v0, 1
syscall

# stampa \n (servizio #4)
la $a0, RTN
li $v0, 4
syscall

# exit (servizio #10) i programmi finiscono sempre con exit
li $v0, 10
syscall

```

A seconda di come carichiamo il registro v0 viene fatta un'operazione differente per la syscall. La syscall fa una isr in base al valore caricato in v0.

Infatti carichiamo in a0 i parametri da passare alla isr perché questi registri a0 servono per gli argomenti delle procedure, poi con li mettiamo in v0 il valore dell'interruzione richiesta.

Se vogliamo sommare due vettori cosa cambia?

Nella parte dichiarativa aggiungo un altro vettore, poi una variabile per mettere la somma ma sarebbe long perché senno posso sommare pochi elementi.

Poi nel main metterei in t1 il secondo puntatore e quindi nel ciclo in t2 l'elemento da caricare del primo vettore e in t3 l'elemento del 2 vettore.

Istruzioni di tipo J

op	offset
6 (31:26)	26(25:0)

J type = 26-bit immediate operand

- Le istruzioni di tipo J vengono usate per codificare i **salti incondizionati** ad un'istruzione posta all'indirizzo *label*: tale indirizzo è ottenuto dall'assemblatore concatenando opportunamente l'offset al PC.
- L'indirizzo dell'istruzione a cui saltare è dato da $A = PC(31:28) | 4 * \text{offset}$ (il valore $4 * \text{offset}$ viene giustapposto ai primi 4 bit del PC)



0	0	0xc
6 (31:26)	20(25:6)	6 (5:0)

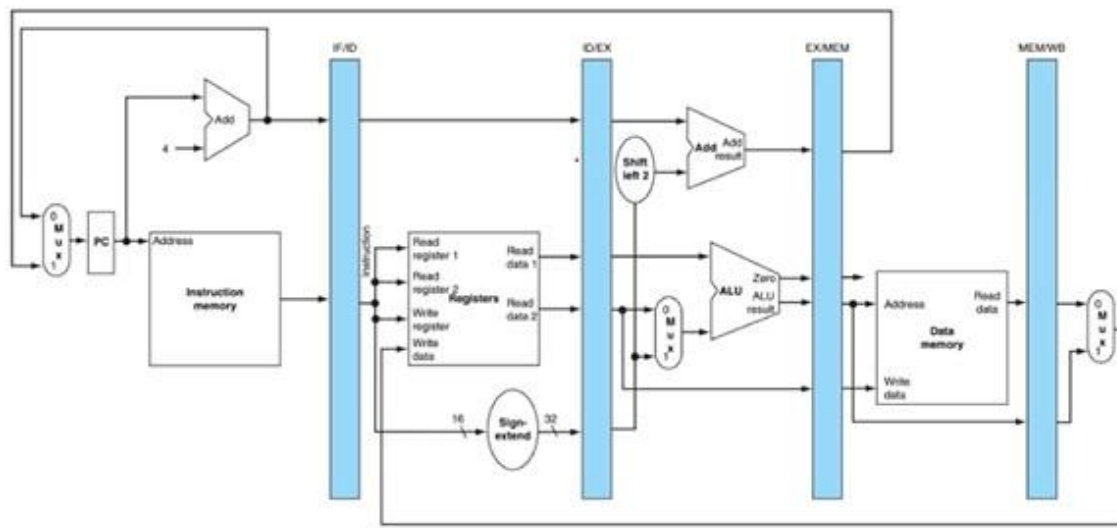
- Il registro \$v0 fornisce il numero della system call: quando si esegue il comando **syscall** viene eseguito il codice relativo alla system call corrispondente all'identificativo posto in \$v0

Service	Code in \$v0	Arguments	Results
print_int	1	\$a0 = integer to be printed	
print_float	2	\$f12 = float to be printed	
print_double	3	\$f12 = double to be printed	
print_string	4	\$a0 = address of string in memory	
read_int	5		integer returned in \$v0
read_float	6		float returned in \$v0
read_double	7		double returned in \$v0
read_string	8	\$a0 = memory address of string input buffer \$a1 = length of string buffer (n)	
sbrk	9	\$a0 = amount	address in \$v0
exit	10		

Riepilogo modalità di indirizzamento

- Il MIPS ammette i seguenti modi di indirizzamento:
 - **Diretto a registro**: l'operando (dato o indirizzo) è contenuto in un registro, il cui identificativo è specificato nell'istruzione. Es: `add $s1, $s2, $s3`
 - **Immediato (tipo I)**: l'operando è una costante il cui valore è specificato nell'istruzione. Es: `addi $s1, $s1, 4`
 - **Base + spiazzamento (tipo I)**: l'operando è in una locazione di memoria il cui indirizzo si ottiene sommando un offset costante ad un indirizzo base contenuto in un registro. Es: `lw $t0, 32($s3)`
 - **Relativo al PC (tipo I)**: l'operando è in una locazione di memoria il cui indirizzo si ottiene sommando un offset al program counter. Es: `bne $s1,$s2,100`
 - **Pseudo-diretto (tipo J)**: parte dell'operando (indirizzo) è specificato come costante nell'istruzione (immediato di 26 bit), e la restante parte è data dal PC. Es: `j 32`
- Il modo di indirizzamento *base + spiazzamento* è l'unico realizzato dalla macchina hardware, mentre gli altri sono realizzati dalla macchina virtuale: l'assemblatore li sintetizza producendo una o più istruzioni per calcolare l'indirizzo effettivo

2.2 Pipeline del MIPS



Il MIPS presenta una pipeline di 5 stadi. Le parti azzurre sono i registri di interfacciamento tra gli stadi. Quindi tra fetch e decode abbiamo il registro IF/ID ecc. questi registri servono ad isolare ogni stadio dal successivo, salvando i risultati parziali al termine di ogni ciclo di clock.

- 1) **IF: prelievo dell'istruzione.** Il PC contiene l'indirizzo dell'istruzione corrente e punta all'istruzione memory. La memoria restituisce l'istruzione a 32 bit che viene salvata nel registro IF/ID. contemporaneamente il sommatore calcola il nuovo indirizzo come PC+4, poiché ogni istruzione occupa 4 byte essendo word-aligned. Infatti il PC oltre che andare in memoria viene anche mandato nel sommatore così da fare quell'operazione per l'istruzione successiva. Il nuovo PC però non va direttamente nella memoria istruzioni ma entra in un mux insieme all'istruzione calcolata a causa di un salto, poiché la prossima istruzione o è quella dopo, o è quella dovuta ad un salto. Questa fase passa alla fase successiva anche il PC.
- 2) **Fase di decodifica:** l'istruzione viene decodificata per capire che tipo di istruzione è ovvero R, I, J type. I campi dell'istruzione vengono usati per leggere i dati dai registri e come sappiamo abbiamo un registro destinazione e massimo due sorgenti. Infatti nel blocco dei registri abbiamo che entrano dal registro di interfacciamento 2 read register (quelli sorgenti), 1 write register (il destinatario), 1 write data che è il valore del risultato ed escono i due valori letti dai registri sorgenti e passa questi valori alla fase EX tramite il registro di interfacciamento. Inoltre passa al registro ID/EX anche il PC passato dalla fase di fetch. Inoltre è presente l'unità sign extend: questa serve nel caso di un salto quando usiamo un valore immediato a 16 bit. Lo estende a 32 bit per essere compatibile con l'ALU.
- 3) **EX fase di esecuzione:** avvengono le operazioni aritmetiche e logiche. Se si tratta di un'istruzione aritmetica utilizza i due valori dei registri e calcola il risultato che mette nel registro EX/MEM. Se invece è un'operazione di load o store somma il valore del registro base all'offset esteso calcolato dall'unità di sign-extend della fase precedente. Infatti prima dell'ALU c'è un mux che se è un'operazione aritmetica fa passare il secondo registro come operando, mentre se è una load/store il MUX fa passare l'offset esteso. Soltanto nella fase EX capiamo che è un salto.

La ALU produce due valori: il risultato dell'operazione e un flag per i salti.

In questa architettura si evidenziano due memorie: dati e istruzioni.

Alla memoria dati accediamo nella fase MEM. Come vediamo c'è un MUX finale per l'operand forwarding che scrive nel registro il dato che proviene o dall'ALU o dalla memoria.

Nel program counter può entrare o il program counter che si incrementa senza salto quindi sommando 4 e l'altro ingresso è quello calcolato se c'è un salto.

Nel caso di un salto viene calcolato l'indirizzo a cui saltare. Questo viene calcolato come $PC+4+4*OFFSET$ grazie all'add nella fase EX

Esempio salto:

beq r1, r2, offset.

- La fase di IF preleva R1 e R2 dai registri interni. La parte immediata dell'istruzione (l'offset del salto, tipicamente a 16 bit) passa attraverso il blocco **Sign-extend**, che la converte a 32 bit.

I valori dei registri, l'offset esteso a 32 bit e il valore $\$PC + 4\$$ avanzano nel buffer **ID/EX**.

- Si passa nella fase EX: l'offset esteso passa per il blocco **Shift left 2**. Questa operazione sposta i bit a sinistra di due posizioni (moltiplicando di fatto per 4) per allineare l'indirizzo

alla word di memoria. Il risultato entra nel sommatore dedicato (Adder in alto), dove viene sommato al $\$PC + 4\$$.

- I due operandi letti nel ciclo precedente entrano nell'**ALU**. L'ALU esegue una sottrazione hardware; se i due valori sono identici, il risultato è zero, portando all'asserzione del flag **Zero** (impostato a 1).
- L'indirizzo di salto calcolato, insieme al segnale Zero e ai dati eventualmente necessari, passano al buffer **EX/MEM**.
- Il diagramma mostra l'indirizzo calcolato che torna indietro tramite la linea di feedback superiore. Se l'istruzione è un salto e la condizione è verificata (flag **Zero** = 1), la logica di controllo (non interamente disegnata qui, ma implicita) attiverà il multiplexer (**Mux**) situato all'estrema sinistra prima del PC

Nota architetturale sui conflitti (Control Hazards): > Dal momento che l'aggiornamento del PC in questo schema avviene solo nello stadio MEM, il processore continuerà ciecamente a caricare nuove istruzioni nei cicli precedenti. Se il salto viene effettivamente "preso", le istruzioni che nel frattempo hanno riempito gli stadi IF, ID ed EX contengono le istruzioni del ramo sbagliato. Il processore dovrà eseguire un *flush* (svuotamento) della pipeline per annullarle e ricominciare il fetch dal nuovo indirizzo corretto.

Meccanismi di gestione:

- Si procede nell'ordine di caricamento sequenziale delle istruzioni salvo ricominciare da capo con le istruzioni giuste se ci si accorge di un conflitto (si sprecano cicli di clock)
- Si adotta una predizione statica dei salti (es. tutti i salti indietro vengono presi e tutti quelli in avanti no)
- Si adotta la tecnica del **delayed branch**, che consiste nel piazzare dopo un branch (nel delay slot) una istruzione che è comunque necessario eseguire, indipendentemente dall'esito del branch. L'istruzione viene quindi portata a termine e solo a quel punto il salto viene (o meno) eseguito.

- 4) **FASE MEM**: Questo stadio è dedicato esclusivamente all'interazione con la memoria dati (Data Memory). Il comportamento dipende strettamente dal tipo di istruzione:

- **Load (lw)**: La memoria viene letta. L'indirizzo da cui leggere è quello appena calcolato dall'ALU nello stadio EX e per questo entra in ADDRESS. Il dato letto viene passato al registro di pipeline MEM/WB.
- **Store (sw)**: Il dato deve essere scritto in memoria. Il dato da scrivere è il contenuto del secondo registro (letto nello stadio ID e fatto scivolare intatto fino a qui). L'indirizzo di destinazione è, di nuovo, quello calcolato dall'ALU.
- **Istruzioni R-type (es. add, sub)**: La memoria dati non viene minimamente toccata. Il risultato matematico calcolato dall'ALU nello stadio EX si limita a "viaggiare" attraverso questo stadio per arrivare al buffer MEM/WB.

- 5) **WB (Write Back)**: L'ultimo stadio ha il compito di scrivere il risultato finale nel banco dei registri (Register File).

- All'estrema destra del datapath c'è un ultimo multiplexer. Il suo compito è scegliere cosa scrivere nel registro di destinazione.
- Se l'istruzione era una **Load**, il MUX sceglie il dato proveniente dalla Data Memory.
- Se l'istruzione era di tipo **Aritmetico/Logico**, il MUX sceglie il risultato proveniente dall'ALU.
- Per istruzioni come **Store** o **Branch**, in questa fase non viene scritto nulla nel banco dei registri (il segnale di controllo *RegWrite* è a 0).

Per risolvere il potenziale conflitto sul register file, le operazioni di lettura (decode) e scrittura (write back) vengono effettuate in due intervalli separati dello stesso ciclo di clock:

- la fase di decode opera nella seconda parte del ciclo di clock
- la fase di write back opera nella prima parte del ciclo di clock

Per l'operand forwarding Il risultato della ALU viene reso immediatamente disponibile usando i registri intermedi della pipe.

Non risolve il conflitto in caso di load: il MIPS nelle sue versioni iniziali adottava il **delayed load**, che consiste nell'inserire in fase di compilazione istruzioni che non generano conflitto subito dopo una load (load delay slot)

Esempio

```
add  r3, r1, r2
nand r6, r4, r5
lw   r4, 20(r2)
add  r5, r2, r5
sw   r7, 12(r3)
```

